Technical White Paper: Forward Automatic Differentiation in faust-rs

OpenAI Codex

2026-04-24

Forward Automatic Differentiation in `faust-rs`

Executive Summary

`faust-rs` now exposes a native forward automatic differentiation primitive, `fad(expr, seed)`, inside the compiler pipeline. This turns differentiation into a compile-time symbolic transform rather than a dynamic runtime graph.

The practical result is that a Faust DSP program can now emit both its primal signal and exact local derivatives with respect to explicit seed parameters, while keeping the normal static Faust compilation model:

- parsing and evaluation stay symbolic,
- propagation builds a differentiated signal graph,
- FIR lowering and backends emit ordinary DSP code,
- recursion is handled without requiring an external ML runtime.

This is not reverse-mode machine learning infrastructure retrofitted onto DSP. It is a forward-mode differentiable extension of the Faust signal language, implemented directly in the Rust compiler.

1. Overview

`faust-rs` now exposes a forward-mode automatic differentiation primitive, `fad(expr, seed)`, inside the Faust language accepted by the Rust compiler. It lets a DSP author ask for the value of an expression and its derivative with respect to one or more explicit seed signals, without manually rewriting the signal graph into dual form.

At a high level:

- `expr` is the DSP expression to differentiate.
- `seed` selects the variable or variables with respect to which the derivative is taken.
- the result is a multi-output DSP expression:
  - first the primal value of `expr`,
  - then one tangent output per seed.

The primitive is deliberately explicit. `faust-rs` does not implicitly differentiate with respect to "all controls in scope"; instead, the seed expression determines the differentiation variables. This makes the primitive predictable, composable, and much easier to reason about in recursive code.

The current implementation focuses on **forward AD**. Reverse AD (`rad`) is not part of the supported compilation path yet.

This note summarizes:

1. the surface semantics of `fad(expr, seed)`,
2. the compiler pipeline used to implement it,
3. the recursion model and why it matters,
4. the supported boundaries of the feature,
5. practical DSP usage patterns with examples.

2. Surface Semantics

2.1 Basic rule

Conceptually:

```
fad(expr, seed) = [ expr, d(expr)/d(seed_0), d(expr)/d(seed_1), ... ]
```

If `seed` is a single signal, `fad` produces two outputs:

- primal,
- tangent.

If `seed` is a tuple-like parallel expression, `fad` produces one tangent per seed lane. For example:

```
process = fad(x * y, (x, y));
```

returns three outputs:

1. `x * y`
2. `d(x*y)/dx = y`
3. `d(x*y)/dy = x`

2.2 Tangent extraction

Because `fad` returns the primal first, a common pattern is to select only the derivative lane:

```
grad = fad(loss, param) : !, _;
```

This keeps the tangent and discards the primal. In the current output ordering, `!, _` is the standard "tangent-only" projection for a single-seed `fad`.

2.3 Multi-seed behavior

Multi-seed `fad` is observable in output order:

```
f = hslider("freq", 1000, 50, 4000, 1);
q = hslider("q", 1.0, 0.1, 10.0, 0.01);
x = no.noise;

process = fad(x : fi.resonlp(f, q, 1.0), (f, q));
```

This produces:

1. the filter output,
2. the derivative with respect to `f`,
3. the derivative with respect to `q`.

The implementation preserves this ordering throughout propagation and code generation.

3. Why `fad` Matters in DSP

Forward AD is useful whenever a DSP patch needs local sensitivity information. Typical examples are:

- gradient monitoring for adaptive algorithms,
- parameter estimation,
- in-graph optimization and adaptive updates,
- differentiable control analysis,
- recursive adaptive updates where the gradient of a loss depends on the previous state.

In a Faust setting, `fad` is attractive because it operates directly on the signal graph. A user does not need to manually define derivative versions of `sin`, `pow`, `delay`, or recursive projections; the compiler applies the chain rule structurally.

This makes small differentiable DSP programs much easier to write and maintain than hand-written dual graphs.

4. How `fad` Is Implemented

The implementation follows the normal `faust-rs` pipeline:

```
parse -> boxes -> eval -> propagate -> normalize -> transform -> fir -> backend
```

4.1 Parser and box layer

At parse time, `fad(expr, seed)` is recognized as a dedicated box node. The relevant surface is in:

- crates/boxes/src/builder.rs
- crates/boxes/src/matcher.rs

The parser builds a `ForwardAD(expr, seed)` box wrapper rather than expanding the derivative immediately. That is important because the rest of the compiler still needs to flatten modules, substitutions, metadata, and recursive aliases before signal-level differentiation is safe.

4.2 Evaluation

The evaluator preserves the `ForwardAD` wrapper instead of trying to interpret it as an ordinary computation. This keeps `fad` visible until signal propagation.

The same area also contains the recursion alias preservation work that made patterns such as:

```
state = next ~ _;
prev  = state;
next  = saturate(prev + input);
```

semantically usable inside `faust-rs`. In this document, the safe claim is:

- this pattern has a clear causal DSP meaning,
- `faust-rs` supports it through targeted recursive-alias preservation,
- it should not be presented as a blanket claim about every Faust compiler or every equivalent source form.

That matters because many useful AD examples refer to the previous recursive state by name.

4.3 Propagation and signal differentiation

The actual AD transform lives in:

- crates/propagate/src/forward_ad.rs
- crates/propagate/src/lib.rs

This is the key stage. Once the box graph has been lowered to signals, `generate_fad_signals_multi(...)` invokes a `ForwardADTransform` that traverses the signal DAG and produces a bundle:

```
[primal, tangent_0, tangent_1, ... tangent_n]
```

for every visited node.

The transform is memoized. Shared sub-expressions are differentiated once and reused, which keeps the derivative graph linear in the size of the original DAG.

4.4 Multi-lane dual representation

Earlier prototypes effectively ran one differentiator per seed. The current model carries all tangents together in one bundle:

```
Dual {
  primal,
  tangents[0..N)
}
```

That is especially important for recursive code. Instead of duplicating a recursive shadow for each seed, the transform now builds one interleaved recursive carrier containing:

```
[primal, d/ds0, d/ds1, ...]
```

This reduces redundant state, improves generated code, and keeps tangent order stable.

4.5 FIR and backend code generation

After propagation, `fad` no longer exists as a source primitive. It has become an ordinary expanded multi-output signal graph. The downstream FIR lowering and the C, C++, interpreter, Cranelift, and Wasm backends just see extra outputs and recursive state lanes.

That design is useful because it keeps AD mostly localized to the propagation phase. The backend does not need a dedicated differentiation pass.

5. Recursion Support

Recursion is where AD becomes technically interesting. A purely feed-forward transform is straightforward; recursive DSP requires careful handling of delayed self-reference.

5.1 Local recursive gradients

The following pattern is supported:

```
pan = step ~ _ with {
  target = hslider("Target", 0, -1, 1, 0.01);
  lr = hslider("LR", 0.05, 0, 1, 0.001);

  step(prev) = prev - lr * grad with {
    loss = (prev - target) ^ 2;
    grad = fad(loss, prev) : !, _;
  };
};

process = pan;
```

This is significant because the derivative is consumed locally inside the recursive branch. The compiler now supports that by switching from the original "expand after recursion" model to an **augmented-state recursion** model when a recursive branch consumes `fad` outputs immediately.

5.2 What is still out of scope

The implementation is strong on several recursive families:

- self-recursive state,
- nested recursion,
- mutual recursion,
- multi-output recursion,
- multi-seed recursive differentiation.

However, it is not yet a universal differentiable extension for every Faust construct. In particular:

- `rad(...)` is not supported in the propagation path,
- some external recursive AD feedback patterns still require explicit support work,
- discrete/non-differentiable primitives intentionally fall back to zero tangents.

6. Supported Signal Families

The current forward AD implementation covers the signal families most relevant to DSP:

- arithmetic and algebraic nodes,
- standard transcendentals,
- `pow`, `atan2`, `min/max`, `fmod`, `remainder`,
- delays,
- recursive projections,
- explicit seeds with duplicates or multiple lanes,
- read-only table and waveform reads.

Table reads deserve a special note. `faust-rs` now differentiates read-only `rdtable` and waveform lookups with respect to the index using a symmetric finite-difference slope:

```
d(table[i]) / di ~= (table[i + 1] - table[i - 1]) / 2
```

This is practical for modulation, wavetable indexing, and lookup-based models.

The main intentional "zero tangent" boundaries remain:

- buttons and checkboxes,
- integer-only and bitwise operations,
- casts that destroy differentiable structure,
- mutable table writes and broader effectful memory updates.

7. Demonstration DSP Examples

The white-paper perspective on `fad` is easiest to understand if the use cases are split into three tiers:

1. **Directly supported analysis patches**: feed-forward or recursion-local examples that compile today in the current `faust-rs` subset.
2. **Host-driven adaptive DSP**: the DSP computes losses and gradients, while an external host updates parameters.
3. **More ambitious differentiable systems**: neural or strongly adaptive patches that are architecturally enabled by `fad`, but still depend on a library layer or extra compiler work to become turnkey workflows.

7.1 Single-parameter gain sensitivity

This is the smallest useful example: measure how the output changes with respect to a gain control.

```
os = library("oscillators.lib");

gain = hslider("gain", 0.5, 0, 2, 0.01);
sig  = gain * os.osc(220);

process = fad(sig, gain);
```

Outputs:

1. `gain * osc(220)`
2. `osc(220)`

This is a direct sanity check: the derivative of `gain * x` with respect to `gain` is exactly `x`.

7.2 Exact local slopes for nonlinear DSP

One of the most valuable practical uses of forward AD in DSP is extracting the exact local slope of a nonlinear transfer function. That slope can be reused for analysis, dynamic control, or antialiasing techniques that require derivative information.

```
ma = library("maths.lib");

clipper(drive, x) = ma.tanh(drive * x);
slope(drive, x)   = fad(clipper(drive, x), x) : !, _;

drive = hslider("drive", 2.0, 0.1, 20.0, 0.01);
input = _;

process = clipper(drive, input), slope(drive, input);
```

Outputs:

1. the nonlinear waveshaper output,
2. the exact derivative of the waveshaper with respect to the input.

This is the kind of primitive needed by ADAA-style or local-linearization techniques. Even when the final antialiasing strategy is more elaborate, `fad` provides a compiler-native way to expose the slope without manually deriving and maintaining the formula.

7.3 Two-parameter filter sensitivity

This example exposes both the primal filter output and the sensitivities of the filter with respect to frequency and resonance:

```
fi = library("filters.lib");
no = library("noises.lib");

f = hslider("freq", 1000, 50, 5000, 1);
q = hslider("q", 1.0, 0.1, 10.0, 0.01);
x = no.noise;

process = fad(x : fi.resonlp(f, q, 1.0), (f, q));
```

Outputs:

1. filter output,
2. `d output / d f`,
3. `d output / d q`.

This is a good analysis patch when tuning a filter interactively.

This pattern is also the core of a **grey-box identification** workflow:

- define a parametric DSP block that is still interpretable as a filter,
- compare it to a target behavior,
- expose one gradient lane per parameter,
- update parameters in the host or in a supported recursive optimizer loop.

For example, a host can run a learning loop around the patch above and minimize the distance between a target filtered signal and the current model output.

7.4 Newton-style root finding and implicit DSP equations

Another important class of use cases is solving implicit equations. In analog modeling and zero-delay-feedback style formulations, one often needs both a function value and its derivative with respect to the unknown.

```
ma = library("maths.lib");

error_eq(x, y) = y - ma.tanh(x - y);
newton_step(x, y) = y - (fad(error_eq(x, y), y) : /);

process = _, 0.0 : newton_step;
```

Conceptually, `fad(error_eq(x, y), y)` returns:

1. `F(y)`
2. `F'(y)`

and `:/` computes `F(y) / F'(y)`, so the full step becomes:

```
y_next = y - F(y) / F'(y)
```

This is a compact and expressive way to build Newton-Raphson style solvers in Faust source. In more elaborate patches, several such steps can be chained together inside a loop or a recursive structure.

7.5 Recursive local gradient update

The next example uses `fad` inside a recursive state update:

```
target = hslider("Target", 0, -1, 1, 0.01);
lr     = hslider("LR", 0.05, 0, 1, 0.001);

state = step ~ _ with {
  step(prev) = prev - lr * grad with {
    loss = (prev - target) ^ 2;
```

```
    grad = fad(loss, prev) : !, _;
  };
};

process = state;
```

This is not just a toy. It demonstrates that `faust-rs` can differentiate a loss with respect to the previous recursive state and immediately use that gradient in the same recursive branch.

Architecturally, this is the bridge toward **real-time recurrent learning style behavior**: the derivative is propagated alongside the recursive state instead of being recovered later by backpropagating through stored history.

7.6 Host-driven optimization

For larger adaptive examples, one practical pattern is to keep the gradient inside the DSP but close the optimization loop in the host. The repository contains examples of this style:

- tests/corpus/auto_chorus_stereo_fad_host.dsp
- tests/corpus/auto_wah_fad_host.dsp

The DSP computes:

- primal audio,
- loss,
- one or more gradients,

and the host updates the control parameters externally. This remains a useful deployment pattern for integration with plugin hosts, experiments, and offline training workflows, but it is **not** the main conceptual point of `fad`.

This is one recommended deployment pattern today for:

- auto-tuning filters,
- self-calibrating modulated effects,
- adaptive wah/chorus style processors,
- target-matching or system-identification experiments.

It keeps the DSP side simple and real time safe, while the host handles:

- parameter constraints,
- optimization schedules,
- batching or smoothing of gradients,
- persistence and UI integration.

7.7 Faust-expressed optimizers and learning loops

The more interesting long-term point is that `fad` is not only useful for exporting gradients to a host. It also makes it possible to express the **optimizer itself in Faust code**.

That is exactly the direction illustrated by local repository examples such as:

- `fad_filter3.dsp`
- `fad_pendulum_cello4.dsp`

In those patches, Faust code computes:

- the model output,
- the error signal,
- one or more derivatives via `fad`,
- and the update rule for the trainable state.

So the learning loop can stay inside the DSP graph rather than being split between DSP code and host code.

Faust-expressed adaptive optimizers `fad_filter3.dsp` is a better illustration of this idea than a simple sign-descent patch. It keeps two trainable filter parameters, frequency and resonance, in recursive state and updates them directly in Faust.

It already contains the main ingredients of a practical optimizer:

- multi-parameter `fad(..., (f_curr, q_curr))`,
- explicit gradient extraction,
- gradient clipping,
- smoothed first and second moments,
- epsilon-stabilized normalization,
- bounded parameter projection.

The relevant update law looks like this:

```
raw_grad_f = -err * df;
raw_grad_q = -err * dq;

grad_f = max(-1.0, min(1.0, raw_grad_f));
grad_q = max(-0.1, min(0.1, raw_grad_q));

m_f = grad_f : si.smooth(0.9);
v_f = (grad_f * grad_f) : si.smooth(0.999);

m_q = grad_q : si.smooth(0.9);
v_q = (grad_q * grad_q) : si.smooth(0.999);

f_n = max(20.0, min(20000.0, f_curr - lr_f * (m_f / (sqrt(v_f) + eps))));
q_n = max(0.1, min(10.0, q_curr - lr_q * (m_q / (sqrt(v_q) + eps))));
```

This is close to a compact Faust-native Adam-style optimizer. The important point is not the exact schedule, but the fact that the entire update path is still Faust DSP code:

- `fad` computes the local sensitivity,
- Faust computes the gradient signal,
- Faust computes moment estimates,
- Faust computes the normalized step,
- Faust clamps the updated parameters,
- recursion stores the updated parameter for the next sample.

In other words, the optimization law is itself a DSP program, not an external host-side training loop.

More elaborate in-graph learning architectures `fad_pendulum_cello4.dsp` shows a more ambitious direction. It combines:

- a physical or kinetic generator,
- a small neural-style controller,
- differentiable control prediction,
- and an optimizer written in Faust through `ad.fit_adam(...)`.

The important architectural message is that `fad` can serve as the derivative source inside a broader Faust-defined adaptive system. Once the gradient exists as an ordinary signal, nothing prevents Faust code from:

- smoothing it,
- clipping it,
- applying sign descent,
- feeding it into SGD,
- feeding it into Adam-like stateful updates,
- storing optimizer state in recursion.

That is closer to the true differentiable-DSP vision than a purely host-driven story.

7.8 Higher-level differentiable DSP patterns

The whitepapers also point toward more ambitious use cases, such as:

- grey-box system identification,
- differentiable zero-delay analog models,
- neural or recurrent black-box audio models,

- optimizer libraries built on top of `fad`.

Those directions are technically aligned with the current architecture, but they should be described carefully in terms of present support.

Grey-box system identification This is already realistic with the current compiler. A representative pattern is:

```
fi = library("filters.lib");
os = library("oscillators.lib");

target_f = 1200;
f_guess  = hslider("freq", 300, 50, 5000, 1);
q_guess  = hslider("q", 1.0, 0.1, 10.0, 0.01);
input    = os.osc(500);

target = input : fi.resonlp(target_f, 1.0, 1.0);
model  = input : fi.resonlp(f_guess, q_guess, 1.0);
loss   = (target - model) ^ 2;

process = target, model, fad(loss, (f_guess, q_guess));
```

This emits the target, the current model, the scalar loss, and one gradient per parameter. That gradient can then be consumed either:

- by a host-side optimizer,
- or by a Faust-defined update stage stored in recursive state.

Black-box or neural-style modeling The compiler-side differentiation engine is also compatible with the idea of larger differentiable blocks, including recurrent ones. What is still missing is not the core `fad` transform itself, but the higher-level library layer that would package:

- trainable parameter buses,
- optimizer state,
- reusable dense/RNN/LSTM building blocks,
- convenient parameter-update combinators.

So the correct statement today is:

- **the compiler infrastructure is in place for such systems**, especially for forward-mode, recursive local sensitivities, and Faust-defined adaptive updates,
- **the out-of-the-box library ergonomics are still an area for future work**.

That distinction is important because it keeps the white-paper vision aligned with the current repository reality.

8. Current Limits and Practical Guidance

For day-to-day use, the following guidelines are accurate:

- use `fad(expr, seed)` with explicit seed variables,
- prefer the explicit parallel seed form `(a, b, c)` when differentiating with respect to multiple parameters,
- use `: !, _` to extract the tangent for a single-seed `fad`,
- expect robust behavior on feed-forward graphs and on the recursive families already covered by the corpus,
- use host-driven optimization when integration simplicity matters,
- but keep in mind that Faust-defined update laws are also a core intended use of `fad`, not an edge case.

The most important non-goal to keep in mind is that `fad` is not yet "differentiate all Faust code automatically". It is a strong, well-tested forward AD primitive over an explicitly supported differentiable subset of Faust.

9. Conclusion

`fad` gives `faust-rs` a meaningful differentiable DSP capability inside the language itself. The primitive is explicit, compositional, and deeply integrated with the Rust compiler pipeline:

- parser and boxes preserve the AD node,

- evaluation keeps it structural,
- propagation performs the actual signal differentiation,
- recursion is handled through augmented-state lowering and unified tangent lanes,
- backends consume the result as ordinary expanded DSP outputs.

That makes `fad` useful in two complementary ways:

1. as an analysis tool, to inspect local sensitivities of a DSP graph,
2. as a building block for adaptive or optimization-driven audio systems.

The implementation is already broad enough to support serious experimentation, especially on feed-forward graphs, recursive local gradient patterns, Faust expressed optimizers, and host-driven adaptive effects. The remaining work is mostly about extending the supported differentiable subset further, rather than proving the core model.

Appendix A. Full Listing: `fad_filter3.dsp`

```
import("stdfaust.lib");

// The model loops over TWO variables: (f_prev, q_prev)
process = no.noise : train ~ (_, _) : (!, !, _);

train(f_p, q_p, input) = f_n, q_n, model_out
with {
    // 1. Initialization: 1000 Hz and Q = 1.0 on the first sample
    f_curr = select2(f_p == 0.0, f_p, 1000.0);
    q_curr = select2(q_p == 0.0, q_p, 1.0);

    // 2. Target controls (the sound to imitate)
    t_f = hslider("Target Freq", 1200, 20, 20000, 1);
    t_q = hslider("Target Q", 2.0, 0.1, 10.0, 0.01);
    target = input : fi.resonlp(t_f, t_q, 1.0);

    // 3. The model and the error computation
    model_out = input : fi.resonlp(f_curr, q_curr, 1.0);
    err = target - model_out;

    // 4. Multi-dimensional FAD (derivative extraction)
    fad_outs = fad(input : fi.resonlp(f_curr, q_curr, 1.0), (f_curr, q_curr)) : !, _, _;

    df = fad_outs : _, !;
    dq = fad_outs : !, _;

    // --- 5. SAFE ADAM OPTIMIZER ---

    // Raw gradients (MSE)
    raw_grad_f = -err * df;
    raw_grad_q = -err * dq;

    // CLIPPING: mathematically limit gradient magnitude
    grad_f = max(-1.0, min(1.0, raw_grad_f));
    grad_q = max(-0.1, min(0.1, raw_grad_q));

    // Moment estimates (M = inertia, V = variance)
    m_f = grad_f : si.smooth(0.9);
    v_f = (grad_f * grad_f) : si.smooth(0.999);

    m_q = grad_q : si.smooth(0.9);
    v_q = (grad_q * grad_q) : si.smooth(0.999);

    // Learning rates
```

```
    lr_f = 2.0;
    lr_q = 0.01;

    // Larger epsilon to avoid division by zero
    eps = 1e-3;

    // 6. Parameter updates with the ADAM algorithm
    f_n = max(20.0, min(20000.0, f_curr - lr_f * (m_f / (sqrt(v_f) + eps))))
        : hbargraph("Learned Freq", 20, 20000);

    q_n = max(0.1, min(10.0, q_curr - lr_q * (m_q / (sqrt(v_q) + eps))))
        : hbargraph("Learned Q", 0.1, 10.0);
};
```

Appendix B. Full Listing: `fad_pendulum_cello4.dsp`

```
import("stdfaust.lib");
ad = library("ad.lib");

// ============================================================
// 1. KINETIC ENGINE
// ============================================================
kinematics(phi) = pos, vel, acc
with {
    eq = sin(phi) + 0.3 * cos(2.5 * phi);
    pos = eq;
    vel = fad(eq, phi) : !, _;
    acc = fad(vel, phi) : !, _;
};

// ============================================================
// 2. NEURAL NETWORK
// ============================================================
nn_controller(v, w, b) = cutoff
with {
    ctrl = ad.dense(v, w, b, 0.5);
    cutoff = 200 + (abs(ctrl) * 8000) : min(15000);
};

// ============================================================
// 3. STRING MODEL
// ============================================================
cello_string(freq, acc, trigger, cutoff) = noise_burst : resonance
with {
    delay_len = ma.SR / freq;
    noise_burst = no.noise * trigger : fi.lowpass(1, 2000);

    gain = 0.990 + (ad.sigmoid(acc) * 0.009);
    damping(sig) = sig : fi.lowpass(1, cutoff) * gain;

    resonance = (+ : de.fdelay(4096, delay_len)) ~ damping;
};

body_resonator = fi.highpass(2, 60) : _ <: (f1, f2, f3) :> /(2.5)
with {
    f1 = fi.resonbp(235, 4.0, 1.0);
    f2 = fi.resonbp(430, 3.0, 0.8);
    f3 = fi.resonbp(850, 5.0, 0.5);
```

```
};

// ============================================================
// 4. TRAINING LOOP
// ============================================================
process = train ~ si.bus(2) : !, !, _ <: _, _;

train(w_p, b_p) = w_n, b_n, audio_out
with {
    // --- USER INTERFACE ---
    lfo_speed = hslider("h:[1] Automate/[1] Vitesse", 0.5, 0.1, 5, 0.01);
    base_freq = hslider("h:[2] Instrument/[1] Fondamentale", 110, 55, 880, 0.1);
    pressure = hslider("h:[2] Instrument/[2] Pression", 0.5, 0.0, 3.0, 0.01);
    lr = hslider("h:[3] Reseau/[1] Apprentissage (LR)", 0.001, 0.0001, 0.05, 0.0001);
    bypass_ia = checkbox("h:[5] Comparaison/[1] Bypass IA (Cible Physique)");

    // --- KINETIC ENGINE ---
    phi_gen = (+(lfo_speed / ma.SR) : ma.frac) ~ _ : *(2 * ma.PI);
    kin_bus = kinematics(phi_gen);
    p = kin_bus : ba.selectn(3, 0);
    v = kin_bus : ba.selectn(3, 1);
    a = kin_bus : ba.selectn(3, 2);

    freq = base_freq * pow(2, p);
    trig = max(0.0, a) * pressure;

    // --- CONTINUOUS DEFIBRILLATOR (DITHERING) ---
    // Add inaudible micro-noise so the ADAM gradient never fully dies.
    w_curr = w_p + (no.noise * 0.000001);
    b_curr = b_p + (no.noise * 0.000001);

    // --- DDSP SPLIT ---

    // A. The model predicts the parameter
    curr_cutoff = nn_controller(v, w_curr, b_curr);

    // B. Complex physical target to imitate (with strict min restored)
    target_cutoff = 200 + (abs(a) * 6000) + (abs(v) * 2000) : min(15000);

    // C. Error computation
    err = target_cutoff - curr_cutoff;

    // D. FAD computed on the fully differentiable model
    dw = ad.grad(nn_controller(v, w_curr, b_curr), w_curr);
    db = ad.grad(nn_controller(v, w_curr, b_curr), b_curr);

    // E. A/B test: choose which signal drives the DSP
    active_cutoff = select2(bypass_ia, curr_cutoff, target_cutoff);

    // F. Audio synthesis
    raw_audio = cello_string(freq, a, trig, active_cutoff);
    final_audio = body_resonator(raw_audio);

    // --- VISUALIZATION & ANTI-OPTIMIZATION TRICK ---
    visu_target = target_cutoff : si.smoo : hbargraph("h:[4] Visu/[1] TARGET Physique", 200, 15000);
    visu_nn = curr_cutoff : si.smoo : hbargraph("h:[4] Visu/[2] NN IA", 200, 15000);

    // The inaudible addition forces the C++ compiler to keep the UI visualization path
```

```
    audio_out = final_audio + ((visu_target + visu_nn) * 0.0000000001);

    // --- MODEL UPDATE ---
    w_n = ad.fit_adam(w_p, 0.5, err, dw, lr, -1.0, 1.0);
    b_n = ad.fit_adam(b_p, 0.0, err, db, lr, -1.0, 1.0);
};
```